

MILESTONE 1

Generación.

Las tres señales que excitan la sala — y los tests que las validan.

¿Dónde estamos?

Pasamos del plano al primer código que calcula. Esto es lo que va a usar el resto del TP.

MILESTONE	ENTREGAN	FECHA	PESO
M0 El Plano	Arquitectura · repo · issues	28 Abr	5%
M1 Generación	Ruido rosa · sine sweep · playrec	19 May — hoy	15%
M2 Procesamiento	Filtros · deconvolución · RI	16 Jun	20%
M3 Producto	API REST · informe · demo	7 Jul	30%

CÓMO SE CREA EL TAG — DESDE MAIN, DESPUÉS DEL ÚLTIMO PR MERGEADO

```
git checkout main && git pull origin main           # estar al día con el remote
git tag -a v0.1.0 -m "M1: ruido rosa, sine sweep, playrec"  # tag anotado
git push origin v0.1.0                                     # empujar SOLO ese tag al remote
```

-a = tag **anotado** (con autor, fecha y mensaje). Sin -a es un tag liviano y no queda quién lo hizo. Si no está el tag empujado al remote, oficialmente no entregaron.

Del plano al primer cálculo.

En M0 dibujaron las tres capas. En M1 le ponen **código real** a una sola de ellas: `services/`.

Los routers no se tocan todavía. Los schemas tampoco. Solo:

- `services/pink_noise.py`
- `services/sine_sweep.py`
- `services/audio_io.py` (reproducir/grabar)

Cada uno es una **función pura de DSP**: entra `numpy`, sale `numpy`. No saben de HTTP. No saben de JSON. No saben de FastAPI.

POR QUÉ IMPORTA

- Si la función es pura, los tests son triviales
- Si la función es pura, en M3 la enchufan a una ruta y listo
- Si la función mezcla cosas, en M2 van a sufrir

Regla: el service no abre archivos, no llama a la red, no imprime. Recibe arrays, devuelve arrays.

Tres funciones. Una entrega.

01 / 03 · función
generar_ruido_rosa

Ruido rosa.

IN duracion · float · segundos
fs · int · Hz

OUT np.ndarray · normalizado en [-1, 1]

TEST QUE VALIDA
PSD con pendiente -3 ± 1 dB/oct.

02 / 03 · función
generar_sine_sweep

Sweep + filtro inverso.

IN f1, f2 · float · Hz
duracion · float · segundos
fs · int · Hz

OUT tuple[np.ndarray, np.ndarray]
(sweep, filtro_inverso)

TEST QUE VALIDA
 $\text{sweep} \times \text{inverso} \approx \delta(t) \cdot \text{pico} > 40 \text{ dB sobre piso.}$

03 / 03 · función
reproducir_y_grabar

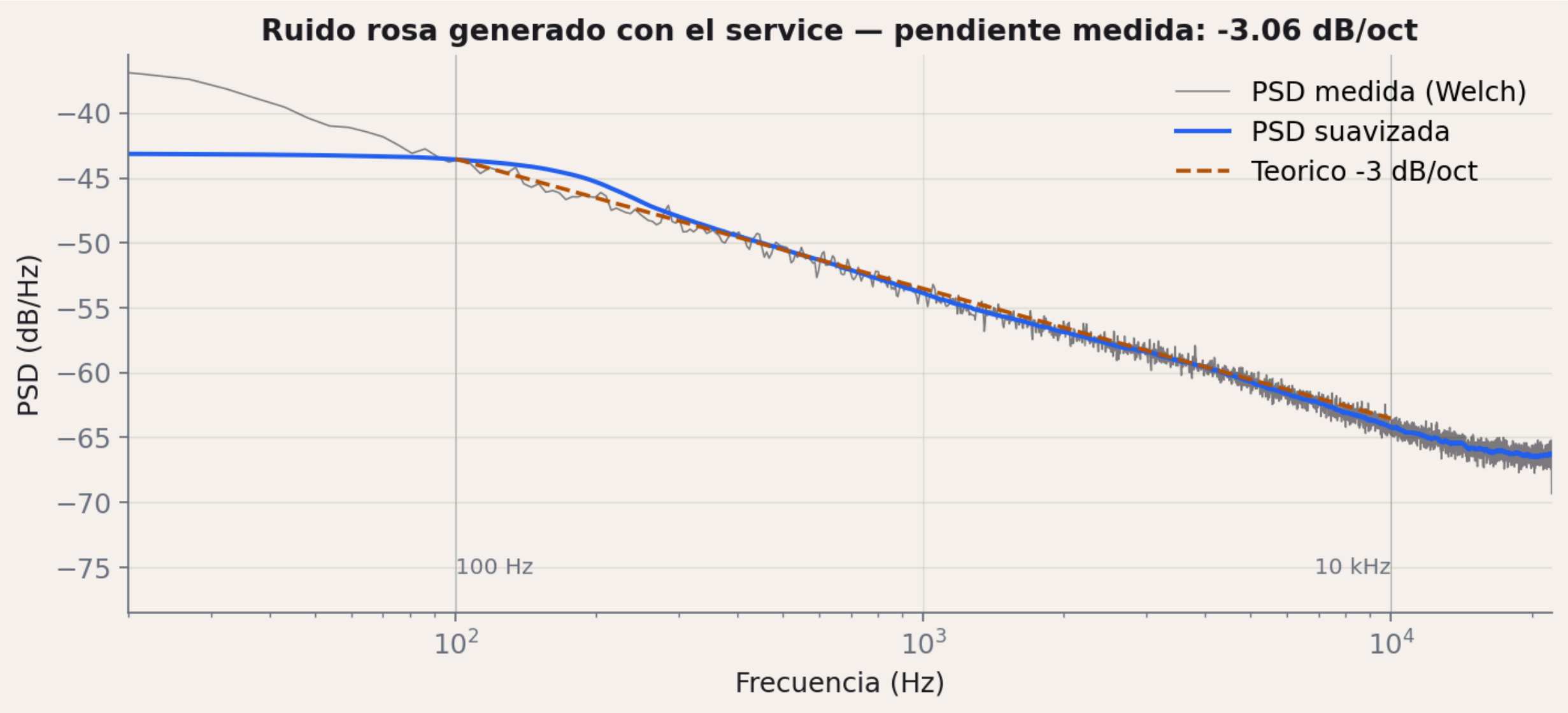
Adquisición.

IN signal · np.ndarray · mono o estéreo
fs · int · Hz
duracion_grabacion · float · s

OUT np.ndarray · señal grabada

TEST QUE VALIDA
Shape correcto · duración $\pm 1\%$ · error si no hay device.

Spec completa en
trabajo_practico/especificacion/m1_generacion.md ·
cambian la firma y los tests dejan de pasar.

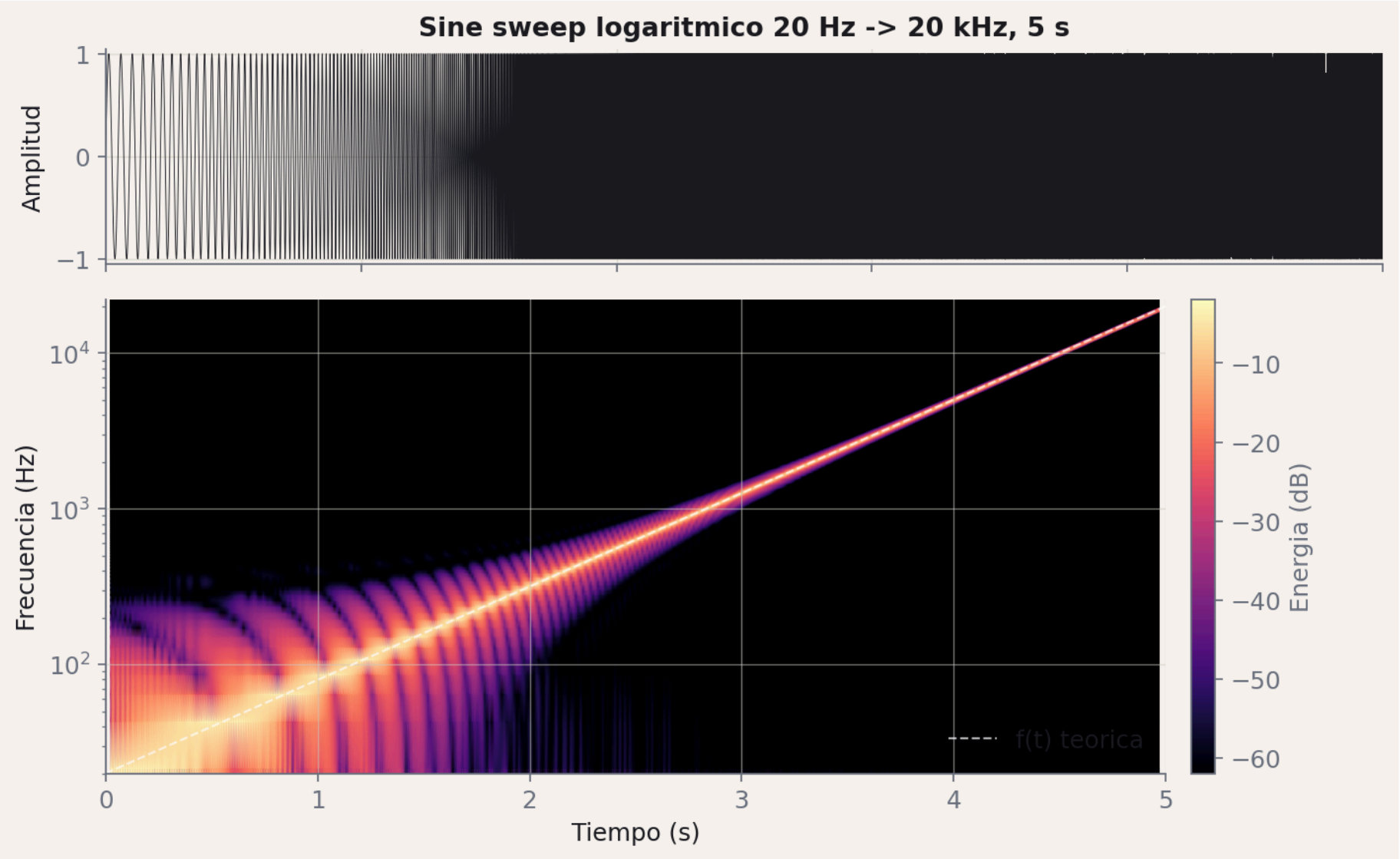


PSD MEDIDA (WELCH) VS. TEÓRICO -3 DB/OCT · PENDIENTE MEDIDA SOBRE LA API DE CÁTEDRA: -3.02 DB/OCT

El ruido rosa (también llamado ruido $1/f$) se caracteriza por tener una densidad espectral de potencia inversamente proporcional a la frecuencia: $S(f) = k/f$, donde k es una constante. En escala logarítmica esto corresponde a una caída de **-3 dB/octava** (o equivalentemente, -10 dB/década), porque $\Delta L = 10 \log_{10}(1/2) \approx -3,01$ dB.

Algoritmo recomendado **Voss-McCartney**: suma de múltiples generadores de ruido blanco que se actualizan a tasas 2^i . El generador i se actualiza cuando el bit i del índice n cambia. La suma produce una señal cuyo espectro se aproxima a $1/f$. Alternativa aceptable: ruido blanco filtrado en frecuencia con $H(f) = 1/\sqrt{f}$.

Spec · m1_generacion.md §1 – [Voss, R. F. & Clarke, J. \(1978\). "1/f noise" in music. JASA 63\(1\).](#)

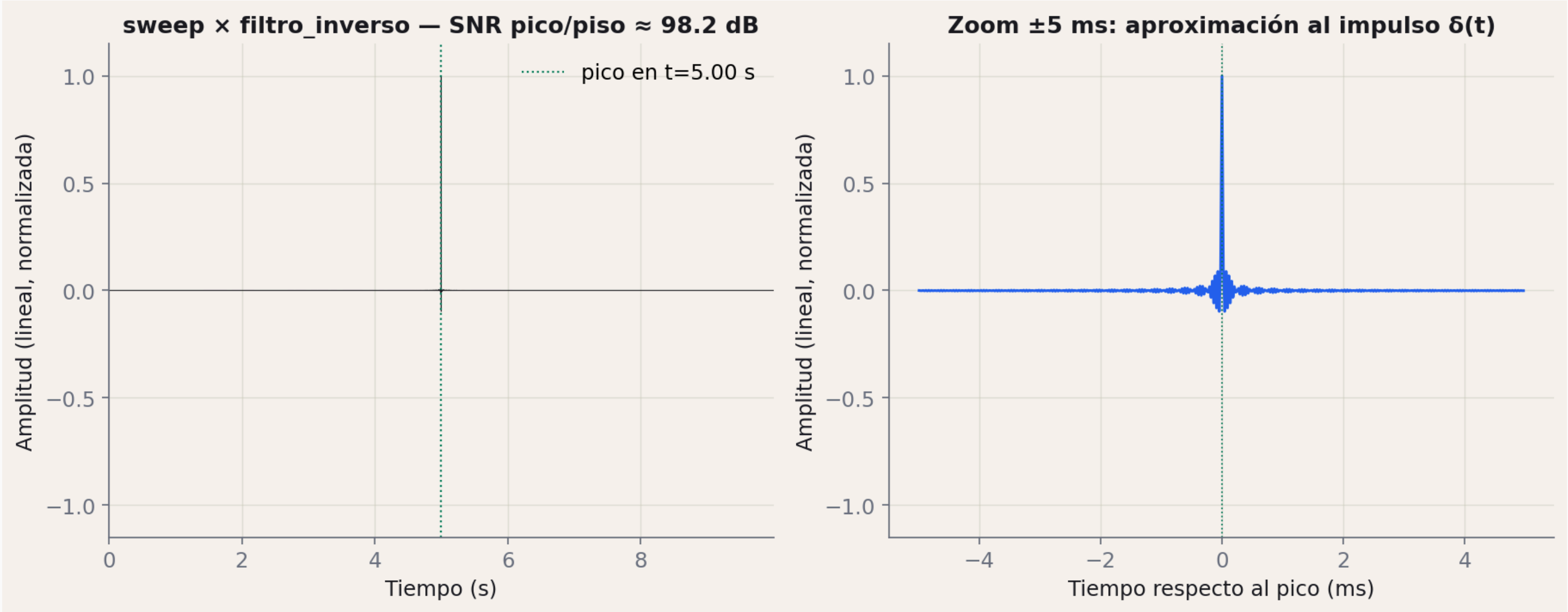


SWEEP 20 HZ → 20 KHZ, 5 S · LÍNEA BLANCA DISCONTINUA = F(T) TEÓRICA

El sine sweep logarítmico (también llamado exponencial) se define como: $x(t) = \sin\left[\frac{2\pi f_1 T}{\ln(f_2/f_1)}\left(e^{t\ln(f_2/f_1)/T} - 1\right)\right]$, con f_1 frecuencia inicial, f_2 final, T duración total, $0 \leq t \leq T$.

La frecuencia instantánea es $f(t) = f_1 \cdot e^{t\ln(f_2/f_1)/T}$, que crece exponencialmente de f_1 a f_2 . Logarítmico (no lineal) porque buscamos [igual energía por octava](#) — coincide con la escala de los filtros de bandas IEC 61260 que van a usar en M2.

Spec · m1_generacion.md §2 – [Farina, A. \(2000\). Simultaneous measurement of impulse response and distortion with a swept-sine technique. 108th AES Convention.](#)



CONVOLUCIÓN SWEEP × INVERSO · PICO VS. PISO ≈ 98 DB DE RELACIÓN (TEST PIDE ≥ 40 DB)

El filtro inverso se obtiene invirtiendo temporalmente el sweep y aplicando una corrección de amplitud que compensa la distribución no uniforme de energía por frecuencia del sweep logarítmico: $x_{\text{inv}}(t) = x(T - t)/A(t)$, con envolvente $A(t) = e^{-t \ln(f_2/f_1)/T}$.

La corrección es necesaria porque el sweep logarítmico permanece **más tiempo en las frecuencias bajas** y concentra más energía allí. El filtro inverso compensa atenuando esa banda. La convolución del sweep con su filtro inverso debe producir un impulso ideal:

$x(t) * x_{\text{inv}}(t) \approx \delta(t)$. En la práctica, un pulso estrecho con lóbulos laterales pequeños.

Spec · m1_generacion.md §2 (filtro inverso) – esto es lo que en M2 usan para deconvolucionar la respuesta al impulso de la sala.

Reproducir y grabar en simultáneo.

BACKEND · SOUNDDEVICE

```
import sounddevice as sd
recorded = sd.playrec(signal, samplerate=fs, channels=1, blocking=True)
sd.wait()
```

Hace play + record a la vez sobre el mismo dispositivo, muestra contra muestra. Documenten en el README la config: dispositivo, canales, fs, buffer size.

DETALLES QUE ROMPEN MEDICIONES

- **Pre-roll** 0.5-1 s — compensa latencia del driver
- **Grabación > señal** — capturar la cola de reverberación
- Manejar **mono y estéreo** sin asumir shape
- Error **informativo** si no hay device

CLIENTE · CÓMO SE HACE LO MISMO EN EL NAVEGADOR (FRONTEND DE CÁTEDRA)

`sd.playrec()` no existe en el browser. La analogía es la **Web Audio API**:

- `navigator.mediaDevices.getUserMedia({audio: {sampleRate: {ideal: fs}}})` — permiso + micrófono
- `new AudioContext({sampleRate})` + `AudioBufferSourceNode` — reproduce la señal de excitación
- `new MediaRecorder(stream, {mimeType: 'audio/webm;codecs=opus'})` — captura el stream
- Conversión a WAV en cliente con `OfflineAudioContext` antes de subir al backend
- `setSinkId()` para elegir salida cuando hay múltiples dispositivos

Mismos cuidados: sample rate explícito, manejo de permisos denegados, latencia entre play y record (acá la introduce el browser, no el driver de audio). Grabar en mono para no duplicar el tamaño del archivo subido al backend.

Los cuatro tests.

El contrato de M1 son los tests. Si pasan, hay nota base. Si no pasan, no.

TEST 1 · ESPECTRO RUIDO ROSA

Welch → pendiente entre 100 Hz y 10 kHz.

Criterio: dentro de -3 ± 1 dB/oct.

TEST 2 · RANGO DEL SWEEP

Espectrograma → energía en f_1 y f_2 .

Criterio: frecuencia instantánea monótona creciente.

TEST 3 · CONVOLUCIÓN → IMPULSO

`fftconvolve(sweep, inv)` → pico.

Criterio: pico ≥ 40 dB sobre la media del resto.

TEST 4 · FORMA DE PLAYREC

Acepta 1D y 2D · duración correcta $\pm 1\%$ · error informativo si no hay device.

Tip: mock del dispositivo para CI.

Lo que va en el informe.

Además de los tests automatizados, cada grupo entrega evidencia visual:

- Espectro del ruido rosa abierto en **Audacity** o REW, con la pendiente visible
- Espectrograma del sweep mostrando el barrido $f_1 \rightarrow f_2$
- Gráfica de la convolución sweep × inverso (debe parecer un impulso)
- Prueba real con altavoz + micrófono (puede ser la PC)

Capturas o PNGs van en `docs/m1/` del repo.

REFERENCIA VISUAL

Comparen contra los plots que mostramos hoy (slides 5, 6, 7). Las pendientes y SNRs deberían estar en el mismo orden.

- Ruido rosa: -3 ± 1 dB/oct
- Sweep: barrido monótono visible
- Convolución: pico claro > 40 dB sobre el piso

Si su resultado se ve muy distinto a las imágenes de cátedra, hay un bug — empiecen por ahí.

Buenas prácticas que ya vimos funcionar.

Patrones aplicados en la API de cátedra. No los pide la rúbrica — pero les ahorran trabajo en M2 y M3.

01 · DEVOLVER DICT, NO SOLO EL ARRAY

Junto al `audio_data`, devolver `file_path`, `sample_rate`, `num_samples`, `max_amplitude`. Mismo dict se serializa como JSON en M3 sin retrabajo.

02 · NORMALIZAR AL 90% DEL MÁXIMO

```
signal = signal / max(|signal|) * 0.9
```

Deja headroom para evitar clipping al guardar WAV o reproducir. Multiplicar por `1.0` y rezar no es estrategia.

03 · DESCARTAR EL TRANSITORIO DEL FILTRO IIR

Si usan `scipy.signal.lfilter`, generar `N + nt60` muestras y tirar las primeras `nt60`. Si no, los primeros milisegundos están sucios.

04 · GENERACIÓN ≠ PLOTTING

`pink_noise.py` genera. `signal_utils.py` grafica. Funciones distintas, archivos distintos. Si mañana cambian el plot, no rompen el generador.

05 · `OUTPUT_PATH` OPCIONAL

Default a `tempfile`, no al cwd del que llama. Tests no ensucian el repo; M3 puede generar WAV efímeros para la response.

06 · DOS IMPLEMENTACIONES, MISMO TEST

Voss-McCartney y filtro IIR son ambos válidos. El test es el contrato — no la implementación. Esto les da margen de optimizar después sin romper nada.

Lo que mira el docente.

CÓDIGO

- ☐ Docstrings NumPy en toda función pública
- ☐ Type hints en parámetros y retorno
- ☐ Tests en `tests/` que pasen al correr `pytest -v`
- ☐ Imports limpios, sin código muerto

GIT

- ☐ Cada función en una rama propia
- ☐ Merge a `main` vía PR con review de otro integrante
- ☐ Historial coherente (no un commit gigante "M1")
- ☐ Tag `v0.1.0` anotado y empujado al remote

RECORDATORIO · CÓMO CORRER LOS TESTS

```
# desde la raiz del repo
uv sync                # asegurar dependencias instaladas
uv run pytest -v        # correr todos los tests con output verboso
uv run pytest -v tests/test_pink.py # correr solo un archivo
uv run pytest -v -k "ruido_rosa"  # correr tests que matchean el nombre
```

`-v` (verbose) muestra cada test con su nombre y si pasó; sin `-v` solo ven puntos. `-k` filtra tests por substring del nombre — útil para correr solo lo que estás tocando. Si un test falla, `pytest --pdb` abre el debugger en el punto exacto de la falla, y `pytest -x` detiene la corrida en el primer fallo.

Checklist de aprobación.

Lo que tiene que estar antes de presentar.

- ☐ `generar_ruido_rosa` con test pasando
- ☐ `generar_sine_sweep` + filtro inverso con tests pasando
- ☐ `reproducir_y_grabar` con test de forma
- ☐ Las 3 gráficas de validación en `docs/m1/`
- ☐ `pytest -v` en verde — los 4 tests requeridos
- ☐ Docstrings y type hints en las 3 funciones públicas
- ☐ PRs mergeados, no commits directos a `main`
- ☐ Tag `v0.1.0` anotado y empujado al remote

Ocho checks. Si están, presentan tranquilos.

Para llevar a casa.

DEL TP

- [API de referencia — Swagger](#)
- Especificación:
`trabajo_practico/especificacion/m1_generacion.md`
- Rúbrica: `trabajo_practico/rubrica.md`

MATERIAL TÉCNICO

- [sounddevice docs](#)
- [scipy.signal.welch](#)
- [Farina \(2000\) — Swept-sine](#)
- [Voss-McCartney](#)
- [Audacity — análisis espectral](#)

PRÓXIMOS PASOS

Ahora, presentan.

Próxima clase: frecuencia y filtros · puerta de entrada a M2.